

COMPILER DESIGN

(22CS2235F)

STUDENT ID: 2420030476

STUDENT NAME: Shaik Harshitha ACADEMIC YEAR: 2026-27

Table of Contents

1. Session 01: Removing white spaces and comments Using LEX Tool..... 1
2. Session 02: Lexer Tokenization Process focusing on keywords using LEX tool 10
3. Session 03: Automating Lexical Analyzer Generation Using LEX Tool..... 21
4. Session 04: Elimination of Left Recursion from given CFG 30
5. Session 05: Implement Program to compute the FIRST set from the CFG 40
6. Session 06: Implement Program to compute the FOLLOW set from the CFG..... 49
7. Session 07: Implement Program for Constructing LL(1) Parsing table 59
8. Session 08: Implement Program for Shift-Reduce parser 64
9. Session 09: Implementation of LR parser 69
10. Session 10: Implement a System to generate and interpret quadruples..... 73
11. Session 11: Implement code optimization technique 78
12. Session 12: Implement a module for generating intermediate code..... 83
13. Session 13: Implement basic error detection and reporting system..... 88

A.Y. 2026-2027 LAB/SKILL CONTINUOUS EVALUATION

S.No	Date	Experiment Name	Pre-Lab (10M)	In-Lab (25M)			Post-Lab (10M)	Viva Voce (5M)	Total (50M)	Faculty Signature
				Program/ Procedure (5M)	Data and Results (10M)	Analysis and Inference (10M)				
1.		Removing white spaces and comments Using LEX Tool								
2.		Lexer Tokenization Process focusing on keywords using LEX tool								
3.		Automating Lexical Analyzer Generation Using LEX Tool								
4.		Elimination of Left Recursion from given CFG								
5.		Program to compute FIRST set for given CFG								
6.		Program to compute FOLLOW set for given CFG								
7.		Construction Of LL(1) Parsing table								
8.		Build Shift-Reduce parser								
9.		Implement of LR parser								
10.		Implement a System to generate and interpret quadruples								
11.		Implement code optimization technique								
12.		Implement a module for generating intermediate code								
13.		Implement basic error detection and reporting system								

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 1:

Experiment Title: Removing white spaces and comments Using LEX Tool.

Aim/Objective:

The aim of this lab session is to introduce students to programming for some small program in a HLL, such as C, that enables them to understand the working of small subset of the Lexical analyser. This familiarity will enable them to build confidence, and understanding of the overall module of the lexical analysis (scanning).

Description:

In this lab session, students will learn about

- To understand how to eliminate white space and comments

Pre-Requisites:

Students should have a basic understanding of the C language, and the ability to program using different constructs of the same.

```
#program 1: Removing white spaces.
%%writefile whitespace.c
#include <stdio.h

int main()
{
   char str[100];
   int i;

   printf("Enter any statement: ");
   fgets(str, sizeof(str), stdin);

   printf("Statement without spaces: ");

   for(i = 0; str[i] != '\0'; i++)
   {
      if(str[i] != ' ')
      {
         printf("%c", str[i]);
      }
   }

   return 0;
}
```

```
!gcc whitespace.c -o whitespace
!./whitespace
```

Output:
Enter any statement: Hello World
Statement without spaces: HelloWorld

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured _____ out of **50**
	Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 2:
Experiment Title: Lexer Tokenization Process focusing on keywords using LEX tool.

Aim/Objective:

The aim of this lab session is to introduce students to programming a scanner for some small program (set of input strings) in a HLL, as C. This familiarity will enable them to kick start into the topic, while comparing their program with an actual one, such as LEX. That would make them aware of their shortcomings in terms of the breadth and depth of the subject of lexical analyser, for a real programming language C.

It is hoped that this will make them improve upon their version, by goading them to follow the industry standards.

Learning Outcomes:

- To understand how to design a lexical analyser for a given language

```
%%writefile program.l
%{
#include <stdio.h>
%}

%%
"int"|"float"|"char"|"double"|"void"|"if"|"else"|"while"|"for"|"return"|"break"|"continue"|"switch"|"case" {
printf("'%s' is a Keyword\n", yytext);
}

[a-zA-Z_][a-zA-Z0-9_]* {
printf("'%s' is an Identifier\n", yytext);
}

[0-9]+ {
printf("'%s' is a Number\n", yytext);
}

[ \t\n] {
/* Ignore whitespace, tabs, and newlines */
}

. {
printf("'%s' is an Unknown Character/Operator\n", yytext);
}
%%
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
int main() {
printf("Enter code snippet to analyze (Press Ctrl+D to stop):\n");
yylex();
return 0;
}

int yywrap() {
return 1;
}
```

```
%%shell
sudo apt-get update && sudo apt-get install -y flex
lex -l program.l
gcc lex.yy.c
./a.out
```

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 3:

Experiment Title: Automating Lexical Analyzer Generation Using LEX Tool.

Aim/Objective:

The aim of this lab session is to make the students conversant with the details of the LEX tool, which is an off-the-shelf tool for building scanners.

Description:

In this lab session, students will learn how to use LEX to build for a given smallish subset of C, or any other language, using the given off-the-shelf tool.

Pre-Requisites:

Students should have a basic understanding of lexical analysis process, C language.

```
%%writefile program.l
%{
#include <stdio.h>
%}

%%
"int"|"float"|"char"|"double"|"void"|"if"|"else"|"while"|"for"|"return"|"break"|"continue"|"switch"|"case" {
printf("'%s' is a Keyword\n", yytext);
}

[a-zA-Z_][a-zA-Z0-9_]* {
printf("'%s' is an Identifier\n", yytext);
}

[0-9]+ {
printf("'%s' is a Number\n", yytext);
}

[ \t\n] {
/* Ignore whitespace, tabs, and newlines */
}

. {
printf("'%s' is an Unknown Character/Operator\n", yytext);
}
%%
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
int main() {
printf("Enter code snippet to analyze (Press Ctrl+D to stop):\n");
yylex();
return 0;
}

int yywrap() {
return 1;
}
```

```
%%shell
sudo apt-get update && sudo apt-get install -y flex
lex -l program.l
gcc lex.yy.c
./a.out
```

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 4:

Experiment Title: Elimination of Left Recursion from given CFG (Context Free Grammar)

Aim/Objective:

The aim of this lab session is to introduce students to solution of the left recursive grammars, and the issue of infinite loop, associated with its top-down parser.

Description:
In this lab session, students will learn about CFG, and remove the problem of left recursion.

Pre-Requisites:
Students should have a basic understanding of top-down parsing, and how it is performed.

```
%%writefile program1.c
#include<stdio.h>
#include<string.h>
#include<ctype.h>

int i, err;
char inp[10];

void E();
void E1();
void T();
void T1();
void F();

int main(){
  i=0, err=0;

  printf("Enter String:");
  scanf("%s", inp);

  E();
  if(strlen(inp)==i && err==0)
    printf("\t\t*****STRING ACCEPTED!!!!!!!!!!!!!!*****\n\n");
  else
    printf("\t\t*****STRING NOT ACCEPTED!!!!!!!!!!!!!!*****\n\n");

  return 0;
}
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
void E(){
  T();
  E1();
}

void E1(){
  if(inp[i]=='+'){
    i++;
    T();
    E1();
  }
}

void T(){
  F();
  T1();
}

void T1(){
  if(inp[i]=='*'){
    i++;
    F();
    T1();
  }
}
```

```
void F(){
  if(isalnum(inp[i])){
    i++;
  }
  else if(inp[i]=='('){
    i++;
    E();

    if(inp[i]==')')
      i++;
    else
      err=1;
  }
  else{
    err=1;
  }
```

```
!gcc program1.c -o program1
!./program1
```

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 5:

Experiment Title: Implement Program to compute the FIRST set from the CFG.

Aim/Objective:

To make the student conversant with the predictive parser (top-down) need to know the FIRST set, in order to parse without backtracking.

Description:

The experiment involves the following steps:

i. Understand the working of a top-down parser.
ii. How is there the need to backtrack in the normal top-down search.
iii. Understand why the given parsing, needs for correct prediction the FIRST set.
iv. How to compute the FIRST set for the different grammar symbols.

Pre-Requisites:

Students should have a basic understanding of top-down search process, and the left-to-right derivation that can be derived from that in parsing, with left side symbol of a production replaced by the string.

```
%%writefile kkr.c
#include <stdio.h>
#include <string.h>

int main() {
char rule[100];

// 1. Accept standard production rule format
printf("Enter production rule (e.g., A->Aa|b): ");
if (scanf("%99s", rule) != 1) return 1;

char lhs = rule[0]; // Left-hand side non-terminal character

// 2. Identify direct left recursion layout (e.g., A -> A...)
if (rule[1] == '-' && rule[2] == '>' && rule[3] == lhs) {
char pipe = strchr(rule, '|');

if (pipe != NULL) {
printf("\nOriginal Rule: %s\n", rule);
printf("\nRules After Eliminating Left Recursion:\n");

printf("%c -> %s%c'\n", lhs, pipe + 1, lhs);
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
// 4. Extract and print the alpha rule: A' -> aA' | e
printf("%c' -> ", lhs);
for (char ptr = &rule[4]; ptr < pipe; ptr++) {
putchar(*ptr); // Prints alpha string characters one by one
}
printf("%c' | e\n", lhs);
} else {
printf("\nFormat Error: Missing option separator pipeline
('|').\n");
}
} else {
printf("\nNo direct left recursion detected in this rule.\n");
}
return 0;
}
```

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured _____out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 6:

Experiment Title: Implement a program to compute FOLLOW Set.

Aim/Objective:

To make the student conversant with the predictive parser (top-down) need to know the FOLLOW set, in order to parse without backtracking.

Description:

The experiment involves the following steps:

v. Understand the situations where there is need to compute the FOLLOW set, for a top-down parser. Why top-down search might need to backtrack, in absence of the FOLLOW set.
vi. How to compute the FOLLOW set.

Pre-Requisites:

Students should have a basic understanding of top-down search process, and the left-to-right derivation that can be derived from that in parsing, with left side symbol of a production replaced by the RHS string.

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 7:
Experiment Title: Implement program for constructing LL(1) parsing.

Learning Outcomes:

- To understand how to construct a LL(1) parsing using a program.

```
%%writefile kkr.c
#include <stdio.h>
#include <string.h>

int main() {
char rule[100];

// 1. Accept standard production rule format
printf("Enter production rule (e.g., A->Aa|b): ");
if (scanf("%99s", rule) != 1) return 1;

char lhs = rule[0]; // Left-hand side non-terminal character

// 2. Identify direct left recursion layout (e.g., A -> A...)
if (rule[1] == '-' && rule[2] == '>' && rule[3] == lhs) {
char pipe = strchr(rule, '|');

if (pipe != NULL) {
printf("\nOriginal Rule: %s\n", rule);
printf("\nRules After Eliminating Left Recursion:\n");

printf("%c -> %s%c'\n", lhs, pipe + 1, lhs);
```

```
// 4. Extract and print the alpha rule: A' -> aA' | e
printf("%c' -> ", lhs);
for (char ptr = &rule[4]; ptr < pipe; ptr++) {
putchar(*ptr); // Prints alpha string characters one by one
}
printf("%c' | e\n", lhs);
} else {
printf("\nFormat Error: Missing option separator pipeline ('|').\n");
}
} else {
printf("\nNo direct left recursion detected in this rule.\n");
}
return 0;
}
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
Enter number of productions: 7
Enter productions (use # for epsilon):
E=TR R=+TR R=# T=FY Y=*FY Y=# F=(E) F=i

FIRST:
FIRST(E) = {(}
FIRST(R) = {+#}
FIRST(T) = {(}
FIRST(Y) = {*#}
FIRST(F) = {(}

FOLLOW:
FOLLOW(E) = {$)}
FOLLOW(R) = {$)}
FOLLOW(T) = {+}
FOLLOW(Y) = {+}
FOLLOW(F) = {*}

LL(1) PARSING TABLE
--------------------

E : [E,( -> TR]
R : [R,+ -> +TR] [$,# -> #] [),# -> #]
T : [T,( -> FY]
Y : [Y,* -> *FY] [+,# -> #]
F : [F,( -> (E)]
```

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 8:

Experiment Title: Implement a program for shift-reduce parser.

Learning Outcomes:

- To understand the implementation of shift-reduce parser using a program.

```
%%writefile kkr.c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 20

char prod[MAX][20];
char first[26][20], follow[26][20];
int n;

void add(char *s, char c)
{
    if (!strchr(s, c)) {
        int l = strlen(s);
        s[l] = c;
        s[l + 1] = '\0';
    }
}

int isNT(char c)
{
    return (c >= 'A' && c <= 'Z');
}
void FIRST(char nt)
{
    int i, j;
    for (i = 0; i < n; i++) {
        if (prod[i][0] != nt)
            continue;

        char *rhs = prod[i] + 2;

        if (rhs[0] == '#') {
            add(first[nt - 'A'], '#');
        }
```

```
        else if (!isNT(rhs[0])) {
            add(first[nt - 'A'], rhs[0]);
        }
        else {
            char x = rhs[0];

            if (first[x - 'A'][0] == '\0')
                FIRST(x);

            for (j = 0; first[x - 'A'][j]; j++)
                add(first[nt - 'A'], first[x - 'A'][j]);
        }
    }
}

void FOLLOW()
{
    int i, j, k;

    add(follow[prod[0][0] - 'A'], '$');

    for (k = 0; k < n; k++) {
        for (i = 0; i < n; i++) {
            char A = prod[i][0];
            char *rhs = prod[i] + 2;

            for (j = 0; rhs[j]; j++) {
                if (isNT(rhs[j])) {
                    char B = rhs[j];

                    if (rhs[j + 1] != '\0') {
                        char x = rhs[j + 1];

                        if (!isNT(x))
                            add(follow[B - 'A'], x);
                        else {
```

```
for (int m = 0; first[x - 'A'][m]; m++)
                        if (first[x - 'A'][m] != '#')
                            add(follow[B - 'A'],
                                first[x - 'A'][m]);
                    }
                }
                else {
                    for (int m = 0; follow[A - 'A'][m];
m++)
                        add(follow[B - 'A'],
                            follow[A - 'A'][m]);
                }
            }
        }
    }
}

int main()
{
    int i, j;

    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("Enter productions (use # for epsilon):\n");

    for (i = 0; i < n; i++)
        scanf("%s", prod[i]);

    /* FIRST */
    for (i = 0; i < n; i++)
        FIRST(prod[i][0]);

    /* FOLLOW */
    FOLLOW();

printf("\nFIRST:\n");
    for (i = 0; i < n; i++) {
        char A = prod[i][0];

        if (i == 0 || prod[i - 1][0] != A)
            printf("FIRST(%c) = {%s}\n",
                A, first[A - 'A']);
    }
```

```
printf("\nFOLLOW:\n");
    for (i = 0; i < n; i++) {
        char A = prod[i][0];

        if (i == 0 || prod[i - 1][0] != A)
            printf("FOLLOW(%c) = {%s}\n",
                A, follow[A - 'A']);
    }

    /* LL(1) Parsing Table */
    printf("\nLL(1) PARSING TABLE\n");
    printf("--------------------\n");

    for (i = 0; i < n; i++) {
        char A = prod[i][0];

        if (i > 0 && prod[i - 1][0] == A)
            continue;

        printf("\n%c : ", A);

        for (j = 0; j < n; j++) {
            if (prod[j][0] != A)
                continue;

            char *rhs = prod[j] + 2;

            if (rhs[0] == '#') {
                for (int k = 0; follow[A - 'A'][k]; k++)
                    printf("[%c,# -> #] ",
                        follow[A - 'A'][k]);
            }
            else if (!isNT(rhs[0])) {
                printf("[%c,%c -> %s] ",
                    A, rhs[0], rhs);
            }
            else {
                char B = rhs[0];

                for (int k = 0; first[B - 'A'][k]; k++)
                    if (first[B - 'A'][k] != '#')
                        printf("[%c,%c -> %s] ",
                            A, first[B - 'A'][k], rhs);
            }
        }
    }
    printf("\n");
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
%%shell
gcc kkr.c
./a.out
```

```
Enter number of productions: 7
Enter productions (use # for epsilon):
E=TR R=+TR R=# T=FY Y=*FY Y=# F=(E) F=i

FIRST:
FIRST(E) = {(}
FIRST(R) = {+#}
FIRST(T) = {(}
FIRST(Y) = {*#}
FIRST(F) = {(}

FOLLOW:
FOLLOW(E) = {$)}
FOLLOW(R) = {$)}
FOLLOW(T) = {+}
FOLLOW(Y) = {+}
FOLLOW(F) = {*}

LL(1) PARSING TABLE
--------------------

E : [E,( -> TR]
R : [R,+ -> +TR] [$,# -> #] [),# -> #]
T : [T,( -> FY]
Y : [Y,* -> *FY] [+,# -> #]
F : [F,( -> (E)]
```

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured _____out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 9:
Experiment Title: Implement Program for LR parser.

Learning Outcomes:

- To implement LR Parser.
- Students should have knowledge about bottom-up parsers i.e. Terminology, Reasoning, Properties of LR(k), How LR(0) parsing works.

```
%%writefile lr_parser.c
#include <stdio.h>
#include <string.h>

char stack[100];
int top = -1;

void push(char c)
{
   stack[++top] = c;
   stack[top + 1] = '\0';
}

void pop(int n)
{
   top -= n;
   stack[top + 1] = '\0';
}

int main()
{
   char input[100];
   int i = 0;

   printf("Enter input string: ");
   scanf("%s", input);

   printf("\nSTACK\tINPUT\tACTION\n");

   while (input[i] != '\0') {
```

```
/* Shift */
      if (input[i] == 'i' && input[i + 1] == 'd') {
         push('i');
         push('d');
         i += 2;

         printf("%s\t%s\tSHIFT id\n",
               stack, input + i);

         /* Reduce F -> id */
         pop(2);
         push('F');

         printf("%s\t%s\tREDUCE F->id\n",
               stack, input + i);

         /* Reduce T -> F */
         pop(1);
         push('T');

         printf("%s\t%s\tREDUCE T->F\n",
               stack, input + i);

         /* Reduce E -> T */
         if (input[i] == '\0' || input[i] == '+') {
            pop(1);
            push('E');

            printf("%s\t%s\tREDUCE E->T\n",
                  stack, input + i);
         }
      }
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
else if (input[i] == '+') {
            push('+');
            i++;

            printf("%s\t%s\tSHIFT +\n",
                  stack, input + i);
        }

        else {
            printf("\nString Rejected!\n");
            return 0;
        }
    }

    if (strcmp(stack, "E") == 0)
        printf("\nString Accepted!\n");
    else
        printf("\nString Rejected!\n");

    return 0;
}
```

```
%%shell
gcc lr_parser.c
./a.out
```

```
Enter input string: id+id*id

STACK          INPUT  ACTION
id         +id*id  SHIFT id
F          +id*id  REDUCE F->id
T          +id*id  REDUCE T->F
E          +id*id  REDUCE E->T
E+         id*id   SHIFT +
E+id       *id     SHIFT id
E+F        *id     REDUCE F->id
E+T        *id     REDUCE T->F

String Rejected!
```

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 10:

Experiment Title: Implement a System to generate and interpret quadruples.

Learning Outcomes:

- To implement LALR Parser.
- Students should have knowledge about bottom-up parsers i.e. Terminology, Reasoning, Properties of LR(k), how to construct canonical collection of LR(1) items.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 11:

Experiment Title: Implement a simple code optimization technique.

Learning Outcomes:

- To understand and implement a given high-level programming statement in Quadruple notation using a program

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured_____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 12:

Experiment Title: Implement a module for generating intermediate code.

Learning Outcomes:

- To understand and implement Code Optimization Technique using a program.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured_____ out of **50** Signature of the Evaluator

Experiment	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Session 13:

Experiment Title: Implement basic error detection and reporting system.

Learning Outcomes:

- To understand error detection and reporting system in compiler design.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured_____ out of **50** Signature of the Evaluator